locally on an available thread or remotely on another server. You can decide later where the message is processed and where the actor lives, which is very different than when you're hardcoding threads or using RPC-style networking. Actors make it easy to build an application out of small parts that resemble networked services but have a much smaller footprint and less administrative overhead.

The Akka team took some of the foundational ideas about actors and, in collaboration with other industry leaders working on how to build better systems, synthesized their knowledge into the Reactive Manifesto.

The Reactive Manifesto

The Reactive Manifesto (www.reactivemanifesto.org) is an initiative to push for the design of systems that are more robust, more resilient, more flexible, and better positioned to meet modern demands. The Akka team has been involved in writing the Reactive Manifesto from the beginning, and Akka is a product of the ideas expressed in the manifesto.
Efficient resource usage and an opportunity for applications to automatically scale (also called *elasticity*) are the drivers for much of the manifesto:

- Blocking I/O limits opportunities for parallelism, so nonblocking I/O is preferred.
- Synchronous interaction limits opportunities for parallelism, so asynchronous interaction is preferred.
- Polling reduces the opportunity to use fewer resources, so an event-driven style is preferred.

If one node can bring down all other nodes, that's a waste of resources. So you need isolation of errors (resilience) to avoid losing all your work.
Systems need to be elastic: if there's less demand, you want to use fewer resources. If there's more demand, use more resources, but never more than required.
Complexity is a big part of cost. So if you can't easily test your system, change it, or add new features, you've got a big problem.